

BENNU FINANCE

Guia de Instalacao e Configuracao AWS

Documento Tecnico para Setup do Ambiente de Producao

Versao 1.0 | Fevereiro 2026

Classificacao: Uso Interno - Equipe de TI

Sumario

- 1.** Introducao e Objetivo
- 2.** Pre-requisitos
- 3.** Visao Geral da Arquitetura
- 4.** Etapa 1 - Provisionamento da Infraestrutura
- 5.** Etapa 2 - Banco de Dados (RDS PostgreSQL)
- 6.** Etapa 3 - Containerizacao (Docker + ECR)
- 7.** Etapa 4 - Deploy (ECS Fargate + ALB)
- 8.** Etapa 5 - Autenticacao (Cognito + Google)
- 9.** Etapa 6 - DNS, SSL e CDN
- 10.** Variaveis de Ambiente e Secrets
- 11.** Monitoramento e Alarmes
- 12.** Backup e Restore
- 13.** Deploy de Novas Versoes
- 14.** Checklist de Validacao
- 15.** Troubleshooting
- 16.** Estimativa de Custos
- 17.** Referencias e Links Uteis
- 18.** Glossario

1. Introducao e Objetivo

Este documento orienta a equipe tecnica responsavel pela implantacao do sistema **Bennu Finance** em ambiente de producao na **Amazon Web Services (AWS)**. Ele foi elaborado como complemento ao pacote ZIP contendo o codigo-fonte da aplicacao.

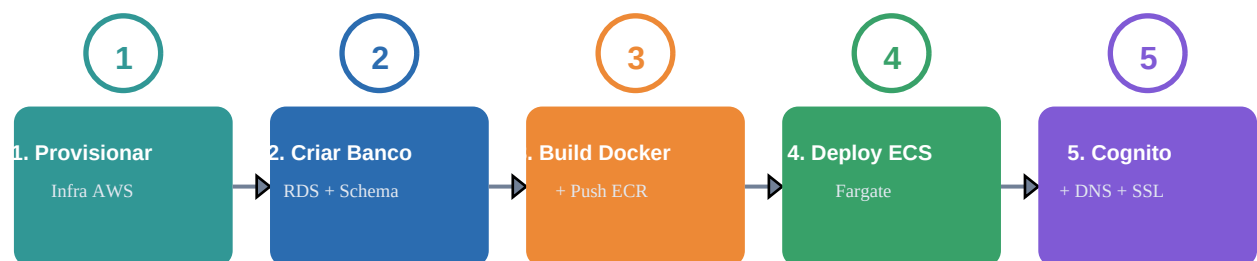
O guia cobre todas as etapas necessarias, desde o provisionamento da infraestrutura ate a validacao final do sistema em producao. Cada etapa esta detalhada com comandos prontos para execucao, diagramas explicativos e notas de contexto para facilitar o entendimento.

Sobre o Bennu Finance

Sistema de gestao financeira multi-empresa desenvolvido em Python (FastAPI) com banco PostgreSQL. Oferece controle de receitas/despesas, categorizacao dual (contabil e gerencial com subcategorias), planejamento orcamentario, relatorios P&L, Cash Flow e Cash Control. Interface web otimizada para telas 1920x1080 com renderizacao server-side (Jinja2 + Bootstrap 5).

Fluxo Geral de Implantacao

A implantacao segue 5 etapas principais, executadas em sequencia:



Tempo estimado: 3 a 5 horas para profissional experiente

Figura 1 - Fluxo sequencial das etapas de implantacao

Publico-Alvo

Profissional de TI com conhecimento basico em AWS (console ou CLI), Docker e PostgreSQL. Nao e necessario conhecimento do codigo-fonte da aplicacao para executar este guia.

2. Pre-requisitos

Antes de iniciar, assegure-se de que as ferramentas abaixo estejam instaladas e configuradas na estacao de trabalho que sera usada para executar os comandos:

Ferramenta	Versao Minima	Finalidade	Como Verificar
AWS CLI	>= 2.x	Comandos de provisionamento AWS	aws --version
Docker	>= 24.x	Build da imagem do container	docker --version
psql (PostgreSQL Client)	>= 14.x	Execucao do script SQL no RDS	psql --version
Python	>= 3.11	Runtime da aplicacao (testes locais)	python --version
Git	>= 2.x	Controle de versao (opcional)	git --version

Tabela 1 - Ferramentas necessarias e versoes minimas

Credenciais AWS

O perfil IAM utilizado deve possuir permissoes para os seguintes servicos:

Servico AWS	Permissoes Necessarias
EC2 / VPC	Criar VPC, subnets, security groups, internet gateway, NAT gateway
RDS	Criar instancias PostgreSQL, subnet groups, snapshots
ECS	Criar clusters, task definitions, services
ECR	Criar repositorios, push de imagens Docker
ELB	Criar Application Load Balancers, target groups, listeners
Cognito	Criar user pools, identity providers, app clients
S3	Criar buckets para backups
CloudWatch	Criar log groups, alarmes, metricas
ACM	Solicitar certificados SSL
Route 53	Gerenciar hosted zones e registros DNS
Secrets Manager	Criar e gerenciar secrets
IAM	Criar roles e policieis para ECS

Tabela 2 - Permissoes IAM necessarias por servico

IMPORTANTE: Nunca utilize a conta root da AWS para operacoes diarias. Crie um usuario IAM dedicado com as permissoes acima e habilite MFA (autenticacao multi-fator).

Estrutura do Pacote ZIP

O ZIP entregue contem a seguinte estrutura de diretorios:

```
bennu-finance.zip
app/ #Codigo-fonte da aplicacao
auth/ # Modulo de autenticacao OAuth/OIDC
middleware/ # Middlewares (auditoria, sessoes)
models/ # Modelos SQLAlchemy (50+ tabelas)
routes/ # Endpoints da API (FastAPI)
services/ # Logica de negocio
static/ # Assets (CSS, JS, imagens)
templates/ # Templates Jinja2 (HTML)
database.py # Configuracao de conexao ao banco
main.py # Entry point da aplicacao
sql/
create_database.sql # Script completo de criacao do banco
pyproject.toml # Dependencias Python
GUIA_INSTALACAO_AWS.md # Versao texto deste guia
MEMORIAL_DESCRITIVO.md # Documentacao funcional completa
```

3. Visao Geral da Arquitetura

O diagrama abaixo ilustra a arquitetura completa do sistema na AWS. Cada componente sera provisionado nas etapas seguintes deste guia.

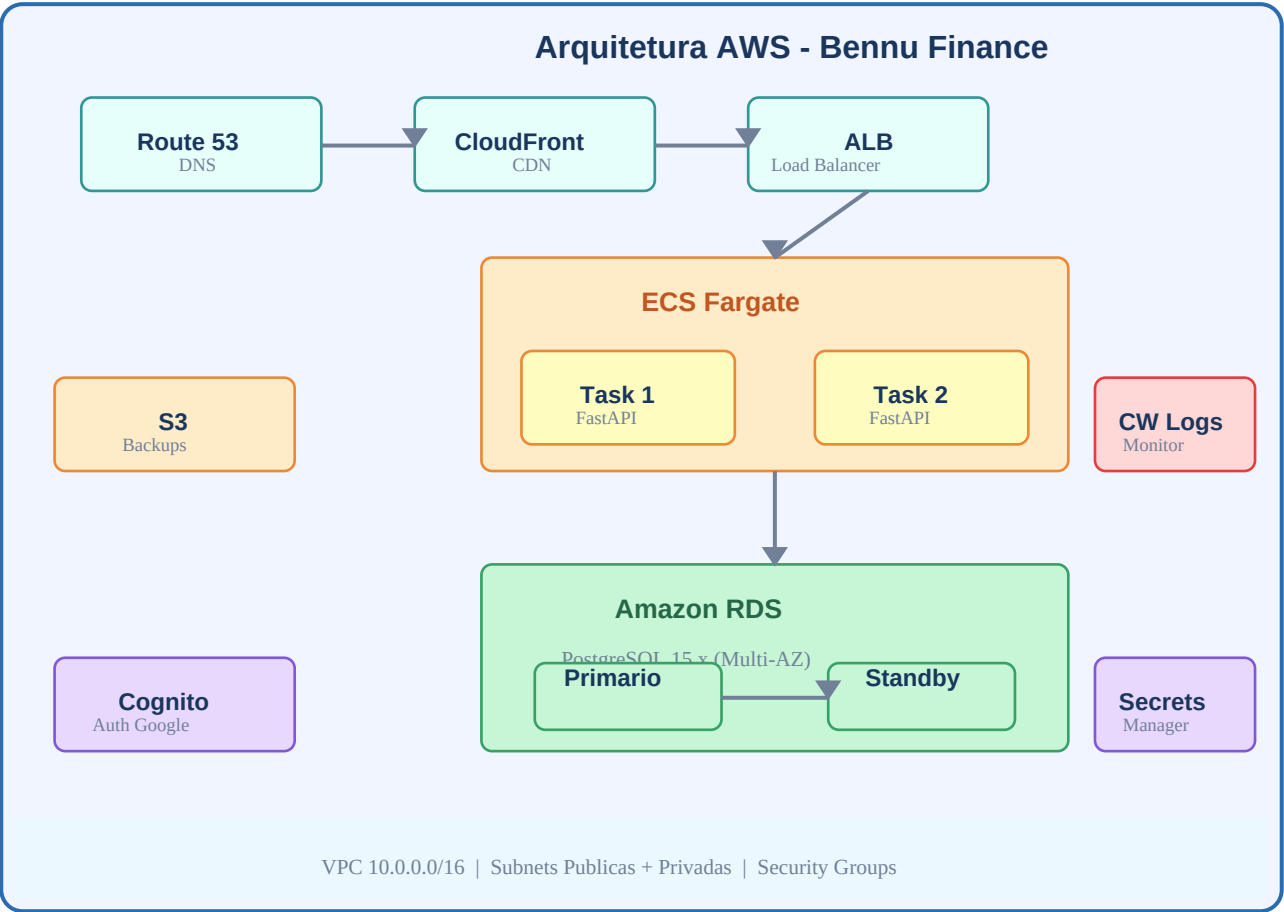


Figura 2 - Arquitetura completa do Bennu Finance na AWS

Descricao dos Componentes

Componente	Servico AWS	Funcao
DNS	Route 53	Resolucao do dominio para o CloudFront/ALB
CDN	CloudFront	Cache e distribuicao de assets estaticos (CSS, JS, imagens)
Load Balancer	ALB	Distribui trafego entre tasks ECS, SSL termination
Aplicacao	ECS Fargate	Executa containers Docker com FastAPI (sem gerenciar servidores)
Banco de Dados	RDS PostgreSQL	Banco gerenciado Multi-AZ com backup automatico
Autenticacao	Cognito	Login Google via OAuth 2.0 / OpenID Connect
Backups	S3	Armazenamento de backups manuais e exportacoes

Componente	Servico AWS	Funcao
Monitoramento	CloudWatch	Logs da aplicacao, metricas de CPU/memoria, alarmes
Credenciais	Secrets Manager	Armazenamento seguro de senhas e chaves de API

Tabela 3 - Componentes da arquitetura e suas funcoes

Diagrama de Security Groups

Os Security Groups controlam o trafego de rede entre os componentes. Funcionam como firewalls virtuais:



Figura 3 - Fluxo de trafego entre Security Groups

NOTA: O principio de **menor privilegio** e aplicado: cada componente so aceita conexoes do componente anterior na cadeia. O banco de dados (RDS) nunca e acessivel diretamente da internet.

4. Etapa 1 - Provisionamento da Infraestrutura

Nesta etapa, criaremos a rede virtual (VPC), subnets, gateways e security groups que formam a base da infraestrutura.

O que é uma VPC?

VPC (Virtual Private Cloud) é uma rede virtual isolada dentro da AWS. Pense nela como a 'rede interna' do seu datacenter, mas na nuvem. Dentro da VPC, criamos subnets (sub-redes) públicas e privadas para separar componentes que precisam ou não de acesso direto à internet.

4.1 Criar a VPC

```
# Criar a VPC com bloco CIDR 10.0.0.0/16 (65.536 IPs disponiveis)
```

```
aws ec2 create-vpc \
--cidr-block 10.0.0.0/16 \
--tag-specifications \
'ResourceType=vpc, Tags=[{Key=Name, Value=bennu-vpc}] '
```

```
# ANOTE o VpcId retornado (formato: vpc-0abc123def456789)
```

DICA: Anote todos os IDs retornados pelos comandos (VPC ID, Subnet IDs, etc). Você precisará deles nas etapas seguintes.

4.2 Criar Subnets

Precisamos de 4 subnets em 2 zonas de disponibilidade (AZs) diferentes para alta disponibilidade:

Subnet	CIDR	AZ	Tipo	Uso
bennu-public-1a	10.0.1.0/24	us-east-1a	Pública	ALB, NAT Gateway
bennu-public-1b	10.0.2.0/24	us-east-1b	Pública	ALB (redundancia)
bennu-private-1a	10.0.3.0/24	us-east-1a	Privada	ECS, RDS
bennu-private-1b	10.0.4.0/24	us-east-1b	Privada	ECS, RDS (redundancia)

Tabela 4 - Configuracao das subnets

```
# Subnets Publicas
```

```
aws ec2 create-subnet --vpc-id <VPC_ID> --cidr-block 10.0.1.0/24 \
--availability-zone us-east-1a \
--tag-specifications 'ResourceType=subnet, Tags=[{Key=Name, Value=bennu-public-1a}] '
```

```
aws ec2 create-subnet --vpc-id <VPC_ID> --cidr-block 10.0.2.0/24 \
--availability-zone us-east-1b \
--tag-specifications 'ResourceType=subnet, Tags=[{Key=Name, Value=bennu-public-1b}] '
```



```
# Subnets Privadas
aws ec2 create-subnet --vpc-id <VPC_ID> --cidr-block 10.0.3.0/24 \
--availability-zone us-east-1a \
--tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=bennu-private-1a}]'
aws ec2 create-subnet --vpc-id <VPC_ID> --cidr-block 10.0.4.0/24 \
--availability-zone us-east-1b \
--tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=bennu-private-1b}]'
```

4.3 Internet Gateway e NAT Gateway

Internet Gateway vs NAT Gateway

Internet Gateway (IGW): Permite que recursos em subnets publicas acessem e sejam acessados pela internet. Exemplo: o ALB precisa receber trafego de usuarios.

NAT Gateway: Permite que recursos em subnets privadas acessem a internet (para baixar atualizacoes, etc) sem serem acessiveis de fora. Exemplo: tasks ECS precisam baixar imagens Docker do ECR.

```
# Internet Gateway
aws ec2 create-internet-gateway \
--tag-specifications 'ResourceType=internet-gateway,Tags=[{Key=Name,Value=bennu-igw}]'
aws ec2 attach-internet-gateway --internet-gateway-id <IGW_ID> --vpc-id <VPC_ID>

# Elastic IP para o NAT Gateway
aws ec2 allocate-address --domain vpc

# NAT Gateway (na subnet publica)
aws ec2 create-nat-gateway --subnet-id <PUBLIC_SUBNET_1A_ID> \
--allocation-id <EIP_ALLOC_ID>
```

4.4 Route Tables

Configure as tabelas de rotas para direcionar o trafego corretamente:

Route Table	Destino	Alvo	Subnets Associadas
bennu-public-rt	0.0.0.0/0	Internet Gateway	bennu-public-1a, bennu-public-1b
bennu-private-rt	0.0.0.0/0	NAT Gateway	bennu-private-1a, bennu-private-1b

Tabela 5 - Configuracao das Route Tables

4.5 Security Groups

Crie os 3 security groups conforme o diagrama da Figura 3:

```
# SG para ALB (acesso publico nas portas 80 e 443)
aws ec2 create-security-group --group-name bennu-alb-sg \
--description 'ALB Security Group' --vpc-id <VPC_ID>
aws ec2 authorize-security-group-ingress --group-id <ALB_SG_ID> \
```

```
--protocol tcp --port 80 --cidr 0.0.0.0/0
aws ec2 authorize-security-group-ingress --group-id <ALB_SG_ID> \
--protocol tcp --port 443 --cidr 0.0.0.0/0

# SG para ECS (acesso apenas do ALB na porta 5000)
aws ec2 create-security-group --group-name bennu-ecs-sg \
--description 'ECS Security Group' --vpc-id <VPC_ID>
aws ec2 authorize-security-group-ingress --group-id <ECS_SG_ID> \
--protocol tcp --port 5000 --source-group <ALB_SG_ID>

# SG para RDS (acesso apenas do ECS na porta 5432)
aws ec2 create-security-group --group-name bennu-rds-sg \
--description 'RDS Security Group' --vpc-id <VPC_ID>
aws ec2 authorize-security-group-ingress --group-id <RDS_SG_ID> \
--protocol tcp --port 5432 --source-group <ECS_SG_ID>
```

5. Etapa 2 - Banco de Dados (RDS PostgreSQL)

O Amazon RDS gerencia o banco de dados PostgreSQL automaticamente, incluindo backups, patches de seguranga e failover Multi-AZ.

O que e Multi-AZ?

Multi-AZ significa que a AWS mantem uma copia sincronizada do banco em outra zona de disponibilidade. Se a zona principal falhar, o failover e automatico (geralmente em menos de 60 segundos). Voce nao precisa fazer nada - a AWS cuida de tudo.

5.1 Criar Subnet Group

```
aws rds create-db-subnet-group \  
--db-subnet-group-name bennu-db-subnet \  
--db-subnet-group-description 'Bennu DB Subnets' \  
--subnet-ids <PRIVATE_SUBNET_1A_ID> <PRIVATE_SUBNET_1B_ID>
```

5.2 Criar Instancia RDS

```
aws rds create-db-instance \  
--db-instance-identifier bennu-finance-db \  
--db-instance-class db.t3.medium \  
--engine postgres --engine-version 15.4 \  
--master-username bennu_admin \  
--master-user-password '<SENHA_FORTE_AQUI>' \  
--allocated-storage 50 --storage-type gp3 \  
--multi-az \  
--db-subnet-group-name bennu-db-subnet \  
--vpc-security-group-ids <RDS_SG_ID> \  
--db-name bennu_finance \  
--backup-retention-period 7 \  
--preferred-backup-window '03:00-04:00' \  
--no-publicly-accessible \  
--storage-encrypted
```

IMPORTANTE: Use uma senha forte (minimo 16 caracteres, com maiusculas, minusculas, numeros e simbolos). Esta senha sera armazenada no Secrets Manager na Etapa 10.

NOTA: A criacao do RDS leva de 5 a 15 minutos. Aguarde o status mudar para 'available' antes de prosseguir.

5.3 Executar Script SQL

Apos o RDS estar disponivel, execute o script de criacao do banco. Como o RDS esta em subnet privada, voce precisara de um **bastion host** ou **VPN** para acessa-lo:

```
# Obter o endpoint do RDS
aws rds describe-db-instances \
--db-instance-identifier bennu-finance-db \
--query 'DBInstances[0].Endpoint.Address' --output text

# Executar script de criacao (via bastion host ou VPN)
psql -h <RDS_ENDPOINT> -U bennu_admin -d bennu_finance \
-f sql/create_database.sql
```

5.4 Migrar Dados Existentes (Opcional)

Se houver dados no ambiente atual que precisam ser migrados:

```
# Exportar dados do ambiente de origem
pg_dump -h <HOST_ORIGEM> -U <USER_ORIGEM> -d <DB_ORIGEM> \
--data-only --no-owner --no-privileges -f dados_export.sql

# Importar no RDS de producao
psql -h <RDS_ENDPOINT> -U bennu_admin -d bennu_finance \
-f dados_export.sql
```

DICA: Execute a importacao em horario de baixo uso. Para bancos grandes, considere usar **pg_dump** com formato custom (-Fc) e **pg_restore** com paralelismo (-j 4).

6. Etapa 3 - Containerizacao (Docker + ECR)

O que e um Container Docker?

Container e uma forma de empacotar a aplicacao com todas as suas dependencias (Python, bibliotecas, etc) em uma 'caixa' portatil. Isso garante que a aplicacao rode exatamente igual em qualquer ambiente. O ECR (Elastic Container Registry) e o 'repositorio' da AWS onde armazenamos essas imagens.

6.1 Criar Dockerfile

Crie o arquivo **Dockerfile** na raiz do projeto com o conteudo abaixo:

```
FROM python:3.11-slim

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    libpq-dev gcc && rm -rf /var/lib/apt/lists/*

COPY pyproject.toml ./
RUN pip install --no-cache-dir .

COPY app/ ./app/

EXPOSE 5000

ENV PYTHONUNBUFFERED=1

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0",
    "--port", "5000", "--workers", "4"]
```

6.2 Criar Repositorio ECR

```
aws ecr create-repository --repository-name bennu-finance \
--image-scanning-configuration scanOnPush=true
```

6.3 Build e Push da Imagem

```
# Login no ECR
aws ecr get-login-password --region us-east-1 | \
docker login --username AWS --password-stdin \
<ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com

# Build da imagem
docker build -t bennu-finance .

# Tag com o endereco ECR
docker tag bennu-finance:latest \
<ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/bennu-finance:latest

# Push para o ECR
docker push \
<ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/bennu-finance:latest
```

DICA: O **ACCOUNT_ID** e o numero de 12 digitos da sua conta AWS. Encontre-o em: `aws sts get-caller-identity --query Account`

7. Etapa 4 - Deploy (ECS Fargate + ALB)

O que e ECS Fargate?

ECS (Elastic Container Service) executa seus containers Docker. Com **Fargate**, voce nao precisa gerenciar servidores - a AWS provisiona e escala a infraestrutura automaticamente. Voce so define quanto de CPU e memoria cada container precisa.

7.1 Criar IAM Role para ECS

O ECS precisa de permissoes para puxar imagens do ECR e ler secrets:

```
# Criar Execution Role
aws iam create-role --role-name bennu-ecs-execution-role \
--assume-role-policy-document '{
"Version": "2012-10-17",
"Statement": [{
"Effect": "Allow",
"Principal": {"Service": "ecs-tasks.amazonaws.com"},
"Action": "sts:AssumeRole"
}]
}'

# Anexar policy padrao do ECS
aws iam attach-role-policy \
--role-name bennu-ecs-execution-role \
--policy-arn arn:aws:iam::aws:policy/service-role/\
AmazonECSTaskExecutionRolePolicy

# Adicionar acesso ao Secrets Manager
aws iam put-role-policy --role-name bennu-ecs-execution-role \
--policy-name SecretsAccess --policy-document '{
"Version": "2012-10-17",
"Statement": [{
"Effect": "Allow",
"Action": ["secretsmanager:GetSecretValue"],
"Resource": "arn:aws:secretsmanager:us-east-1:
<ACCOUNT_ID>:secret:bennu-finance/*"
}]
}'
```

7.2 Criar Task Definition

A Task Definition descreve como o container deve ser executado. Crie o arquivo **task-definition.json**:

Parametro	Valor	Explicacao
cpu	2048 (2 vCPUs)	Capacidade de processamento por task
memory	4096 (4 GB)	Memoria RAM por task
containerPort	5000	Porta onde o FastAPI escuta
desiredCount	2	Numero de tasks rodando simultaneamente
healthCheck	/health	Endpoint que o ALB usa para verificar saude

Tabela 6 - Parametros principais da Task Definition

```
# Registrar task definition
aws ecs register-task-definition \
--cli-input-json file:///task-definition.json
```

NOTA: O arquivo completo task-definition.json esta documentado no [GUIA_INSTALACAO_AWS.md](#) (secao 7.2).

7.3 Criar ALB e Target Group

```
# Criar Application Load Balancer
aws elbv2 create-load-balancer --name bennu-alb \
--subnets <PUBLIC_SUBNET_1A_ID> <PUBLIC_SUBNET_1B_ID> \
--security-groups <ALB_SG_ID> \
--scheme internet-facing --type application

# Criar Target Group
aws elbv2 create-target-group --name bennu-tg \
--protocol HTTP --port 5000 --vpc-id <VPC_ID> \
--target-type ip --health-check-path /health

# Listener HTTPS (porta 443 com certificado SSL)
aws elbv2 create-listener --load-balancer-arn <ALB_ARN> \
--protocol HTTPS --port 443 \
--certificates CertificateArn=<ACM_CERT_ARN> \
--default-actions Type=forward,TargetGroupArn=<TG_ARN>

# Listener HTTP (redireciona para HTTPS)
aws elbv2 create-listener --load-balancer-arn <ALB_ARN> \
--protocol HTTP --port 80 \
--default-actions Type=redirect,RedirectConfig=\
'{Protocol=HTTPS,Port=443,StatusCode=HTTP_301}'
```

7.4 Criar Cluster e Service

```
# Criar cluster ECS
aws ecs create-cluster --cluster-name bennu-cluster

# Criar service (inicia 2 tasks automaticamente)
aws ecs create-service --cluster bennu-cluster \
--service-name bennu-finance-service \
--task-definition bennu-finance --desired-count 2 \
```



```
--launch-type FARGATE \  
--network-configuration 'awsvpcConfiguration={  
subnets=[<PRIV_SUBNET_1A>,<PRIV_SUBNET_1B>],  
securityGroups=[<ECS_SG_ID>],  
assignPublicIp=DISABLED}' \  
--load-balancers 'targetGroupArn=<TG_ARN>,  
containerName=bennu-finance,containerPort=5000'
```

8. Etapa 5 - Autenticacao (Cognito + Google)

O Bennu Finance utiliza autenticacao via Google. Na AWS, o servico **Cognito** gerencia todo o fluxo de login.

Fluxo de Autenticacao

1. Usuario clica em 'Login com Google' no Bennu Finance
2. Cognito redireciona para a pagina de login do Google
3. Usuario autoriza com sua conta Google
4. Google retorna token para o Cognito
5. Cognito valida e redireciona de volta para a aplicacao com sessao ativa

8.1 Configurar Google Cloud Console

- 1 Acesse **console.cloud.google.com** e crie um novo projeto (ex: 'Bennu Finance')
- 2 Va em **APIs & Services > Credentials** e clique em **Create Credentials > OAuth Client ID**
- 3 Tipo: **Web Application**. Nome: 'Bennu Finance'
- 4 Em **Authorized JavaScript Origins**, adicione: `https://<SEU_DOMINIO>`
- 5 Em **Authorized Redirect URIs**, adicione:
`https://bennu-finance.auth.us-east-1.amazoncognito.com/oauth2/idpresponse`
- 6 Anote o **Client ID** e o **Client Secret** gerados

8.2 Criar Cognito User Pool

```
# Criar User Pool
aws cognito-idp create-user-pool \
--pool-name bennu-finance-pool \
--auto-verified-attributes email \
--schema '[{"Name":"email","Required":true,"Mutable":true}]'

# Criar dominio do Cognito
aws cognito-idp create-user-pool-domain \
--user-pool-id <POOL_ID> --domain bennu-finance

# Configurar Google como Identity Provider
aws cognito-idp create-identity-provider \
--user-pool-id <POOL_ID> --provider-name Google \
--provider-type Google --provider-details '{
"client_id": "<GOOGLE_CLIENT_ID>",
"client_secret": "<GOOGLE_CLIENT_SECRET>",
"authorize_scopes": "openid email profile"
```

```
}' --attribute-mapping '{"email":"email","name":"name"}'
```

```
# Criar App Client
```

```
aws cognito-idp create-user-pool-client \
```

```
--user-pool-id <POOL_ID> --client-name bennu-web \
```

```
--generate-secret \
```

```
--supported-identity-providers Google \
```

```
--callback-urls '["https://<SEU_DOMINIO>/auth/callback"]' \
```

```
--logout-urls '["https://<SEU_DOMINIO>/"]' \
```

```
--allowed-o-auth-flows code \
```

```
--allowed-o-auth-scopes 'openid email profile' \
```

```
--allowed-o-auth-flows-user-pool-client
```

9. Etapa 6 - DNS, SSL e CDN

9.1 Certificado SSL (ACM)

```
# Solicitar certificado (validacao via DNS)
aws acm request-certificate \
--domain-name <SEU_DOMINIO> \
--subject-alternative-names '*.<SEU_DOMINIO>' \
--validation-method DNS
```

NOTA: Apos solicitar, a AWS fornecera um registro CNAME para validacao. Adicione esse registro no seu DNS e aguarde a validacao (pode levar ate 30 minutos).

9.2 DNS (Route 53)

```
# Criar registro A apontando para o ALB
aws route53 change-resource-record-sets \
--hosted-zone-id <ZONE_ID> \
--change-batch '{
"Changes": [{
"Action": "CREATE",
"ResourceRecordSet": {
"Name": "<SEU_DOMINIO>",
"Type": "A",
"AliasTarget": {
"HostedZoneId": "<ALB_HOSTED_ZONE_ID>",
"DNSName": "<ALB_DNS_NAME>",
"EvaluateTargetHealth": true
}
}
}]
}'
```

9.3 CloudFront (CDN)

Configure o CloudFront para cache de assets estaticos:

Behavior	Path Pattern	Cache TTL	Compress
Assets estaticos	/static/*	30 dias	Sim (gzip, brotli)
Paginas dinamicas	Default (*)	Sem cache	Sim

Tabela 7 - Configuracao de behaviors no CloudFront

10. Variaveis de Ambiente e Secrets

Todas as credenciais sao armazenadas no **AWS Secrets Manager** e injetadas automaticamente nos containers ECS:

Variavel	Descricao	Exemplo
DATABASE_URL	Connection string PostgreSQL	postgresql://user:pass@host:5432/db
SESSION_SECRET	Chave de assinatura de sessoes	Gerar com: openssl rand -hex 32
REPLIT_DEPLOYMENT	Flag de producao	1
ISSUER_URL	URL do provedor OIDC (Cognito)	https://cognito-idp.us-east-1.amazonaws.com/
COGNITO_CLIENT_ID	Client ID do App Cognito	(fornecido pela AWS)
COGNITO_CLIENT_SECRET	Client Secret do App	(fornecido pela AWS)
COGNITO_DOMAIN	Dominio do Cognito	bennu.auth.us-east-1.amazoncognito.com

Tabela 8 - Variaveis de ambiente obrigatorias

```
# Criar secret no Secrets Manager
aws secretsmanager create-secret \
--name bennu-finance/production \
--secret-string '{
"DATABASE_URL": "postgresql://bennu_admin:<SENHA>@<RDS>:5432/bennu_finance",
"SESSION_SECRET": "<GERAR_COM_openssl_rand_-hex_32>",
"COGNITO_CLIENT_ID": "<ID>",
"COGNITO_CLIENT_SECRET": "<SECRET>",
"COGNITO_DOMAIN": "<DOMAIN>",
"COGNITO_USER_POOL_ID": "<POOL_ID>"
}'
```

IMPORTANTE: Nunca salve senhas ou chaves em arquivos de codigo, repositorios Git ou variaveis de ambiente em texto plano. Sempre use o Secrets Manager.

11. Monitoramento e Alarmes

Configure alarmes no CloudWatch para ser notificado sobre problemas:

Alarme	Metrica	Threshold	Acao
CPU Alta	CPUUtilization (ECS)	> 80% por 10 min	Notificar equipe via SNS
Erros 5xx	HTTPCode_Target_5XX_Count (ALB)	> 10 em 5 min	Notificar equipe via SNS
Memoria Alta	MemoryUtilization (ECS)	> 85% por 10 min	Notificar equipe via SNS
Conexoes DB	DatabaseConnections (RDS)	> 80% max	Investigar connection leaks
Espaco Disco RDS	FreeStorageSpace (RDS)	< 5 GB	Expandir storage

Tabela 9 - Alarmes recomendados

```
# Criar log group para ECS
aws logs create-log-group --log-group-name /ecs/bennu-finance

# Alarme de CPU alta
aws cloudwatch put-metric-alarm --alarm-name bennu-high-cpu \
--metric-name CPUUtilization --namespace AWS/ECS \
--statistic Average --period 300 --threshold 80 \
--comparison-operator GreaterThanThreshold \
--evaluation-periods 2 \
--dimensions Name=ClusterName,Value=bennu-cluster \
Name=ServiceName,Value=bennu-finance-service \
--alarm-actions <SNS_TOPIC_ARN>
```

12. Backup e Restore

Tipo	Frequencia	Retencao	Responsavel
Automatico RDS	Diario (03:00-04:00 UTC)	7 dias	AWS (automatico)
Snapshot Manual	Antes de atualizacoes	Indefinida	Equipe de TI
pg_dump Exportacao	Semanal (recomendado)	30 dias	Equipe de TI

Tabela 10 - Politica de backup recomendada

```
# Backup manual (snapshot RDS)
aws rds create-db-snapshot \
--db-instance-identifier bennu-finance-db \
--db-snapshot-identifier bennu-manual-$(date +%Y%m%d)

# Restore a partir de snapshot
aws rds restore-db-instance-from-db-snapshot \
--db-instance-identifier bennu-finance-db-restored \
--db-snapshot-identifier <SNAPSHOT_ID> \
--db-instance-class db.t3.medium \
--db-subnet-group-name bennu-db-subnet \
--vpc-security-group-ids <RDS_SG_ID>
```

13. Deploy de Novas Versoes

Processo para atualizar a aplicacao em producao:

- 1** **Build** nova imagem Docker: `docker build -t bennu-finance .`
- 2** **Tag** com endereco ECR: `docker tag bennu-finance:latest <ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/bennu-finance:latest`
- 3** **Push** para o ECR: `docker push <ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/bennu-finance:latest`
- 4** **Force new deployment:** `aws ecs update-service --cluster bennu-cluster --service bennu-finance-service --force-new-deployment`
- 5** **Monitorar:** `aws ecs wait services-stable --cluster bennu-cluster --services bennu-finance-service`

NOTA: O ECS realiza **rolling deployment** por padrao: inicia novas tasks antes de parar as antigas, garantindo zero downtime durante a atualizacao.

14. Checklist de Validacao

Apos concluir todas as etapas, valide cada item abaixo:

Infraestrutura

- ☐ VPC criada com 4 subnets (2 publicas, 2 privadas)
- ☐ Internet Gateway e NAT Gateway funcionando
- ☐ Security Groups criados com regras corretas
- ☐ RDS PostgreSQL acessivel pelas tasks ECS

Aplicacao

- ☐ Imagem Docker no ECR com build recente
- ☐ Tasks ECS rodando (desired count = running count)
- ☐ Health check retornando 200 em /health
- ☐ Aplicacao respondendo em https://

Seguranca

- ☐ Certificado SSL valido (sem avisos no navegador)
- ☐ HTTP redirecionando para HTTPS
- ☐ Secrets armazenados no Secrets Manager
- ☐ RDS nao acessivel publicamente
- ☐ Login Google funcionando via Cognito
- ☐ Todas as rotas protegidas exigindo autenticacao

Operacao

- ☐ Logs aparecendo no CloudWatch (/ecs/bennu-finance)
- ☐ Alarmes CloudWatch configurados e testados
- ☐ Backup automatico RDS verificado
- ☐ Assets estaticos carregando (CSS, JS, imagens)

15. Troubleshooting

Guia rapido para resolver os problemas mais comuns:

Problema: 502 Bad Gateway

Causa provavel: Tasks ECS nao estao rodando ou nao passam no health check

Solucao: Verificar: `aws ecs describe-services --cluster bennu-cluster --services bennu-finance-service`.
Consultar logs: `aws logs tail /ecs/bennu-finance`

Problema: Timeout na conexao com o banco

Causa provavel: Security Group do RDS nao permite acesso do ECS

Solucao: Verificar se o SG do RDS tem regra de entrada na porta 5432 com source = SG do ECS

Problema: Login Google nao funciona

Causa provavel: URLs de callback nao conferem

Solucao: Verificar: 1) Callback URL no Cognito, 2) Redirect URI no Google Console, 3) Dominio do Cognito

Problema: Assets nao carregam (CSS/JS)

Causa provavel: Cache do CloudFront desatualizado

Solucao: Executar invalidacao: `aws cloudfront create-invalidation --distribution-id --paths '/static/*'`

Problema: Erro 500 generico

Causa provavel: Erro na aplicacao

Solucao: Consultar CloudWatch Logs: `aws logs tail /ecs/bennu-finance --follow`

Problema: Aplicacao lenta

Causa provavel: CPU ou memoria insuficiente

Solucao: Verificar metricas no CloudWatch. Considerar: aumentar cpu/memory na task definition ou aumentar desired-count

16. Estimativa de Custos

Estimativa de custos mensais para o ambiente de producao basico:

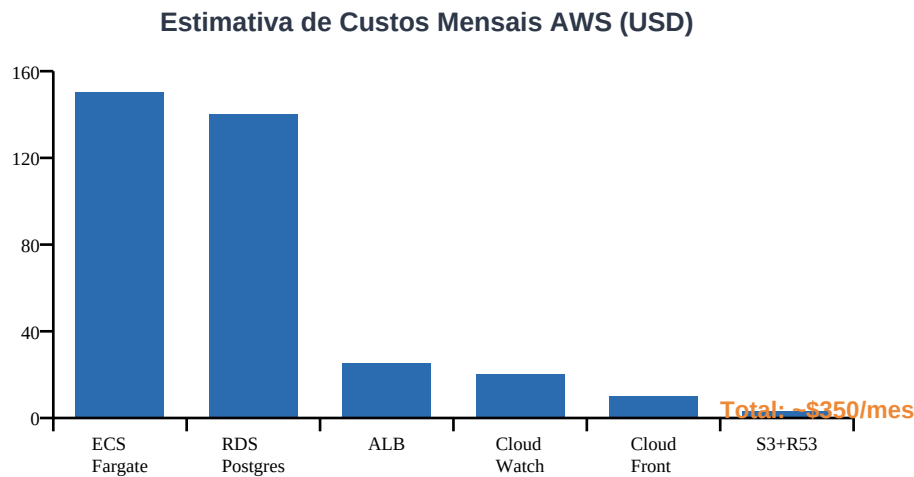


Figura 4 - Distribuicao dos custos mensais por servico AWS

Servico	Especificacao	Custo/Mes (USD)
ECS Fargate	2 tasks x 2 vCPU x 4 GB RAM	~\$150
RDS PostgreSQL	db.t3.medium, Multi-AZ, 50 GB gp3	~\$140
ALB	Application Load Balancer	~\$25
CloudWatch	Logs + metricas + alarmes	~\$20
CloudFront	100 GB transferencia/mes	~\$10
S3	50 GB armazenamento	~\$2
Route 53	1 hosted zone	~\$1
Cognito	Ate 1000 MAU	Gratuito
Secrets Manager	7 secrets	~\$3
TOTAL		~\$350/mes

Tabela 11 - Detalhamento dos custos mensais estimados

NOTA: Valores estimados para a regioao us-east-1 (N. Virginia) em fevereiro/2026. Os custos reais podem variar conforme uso. Consulte [calculator.aws](#) para estimativas atualizadas.

17. Referencias e Links Uteis

Recurso	URL
Documentacao AWS ECS Fargate	docs.aws.amazon.com/ecs/latest/developerguide/
Documentacao AWS RDS PostgreSQL	docs.aws.amazon.com/AmazonRDS/latest/UserGuide/
Guia de VPC e Networking	docs.aws.amazon.com/vpc/latest/userguide/
AWS Cognito Developer Guide	docs.aws.amazon.com/cognito/latest/developerguide/
CloudFront Developer Guide	docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/
AWS Secrets Manager	docs.aws.amazon.com/secretsmanager/latest/userguide/
CloudWatch User Guide	docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/
ACM (Certificate Manager)	docs.aws.amazon.com/acm/latest/userguide/
Route 53 Developer Guide	docs.aws.amazon.com/Route53/latest/DeveloperGuide/
AWS CLI Command Reference	docs.aws.amazon.com/cli/latest/reference/
Docker Documentation	docs.docker.com/
FastAPI Documentation	fastapi.tiangolo.com/
PostgreSQL 15 Documentation	www.postgresql.org/docs/15/
AWS Pricing Calculator	calculator.aws/
Google Cloud Console (OAuth)	console.cloud.google.com/

Tabela 12 - Links de referencia para consulta

18. Glossario

Termo	Definicao
ALB	Application Load Balancer - Distribui trafego entre multiplas instancias/containers
AZ	Availability Zone - Datacenter isolado dentro de uma regio AWS
CIDR	Classless Inter-Domain Routing - Notacao para definir blocos de IPs (ex: 10.0.0.0/16)
CDN	Content Delivery Network - Rede de distribuicao de conteudo para cache global
Cognito	Servico AWS de gerenciamento de identidade e autenticacao de usuarios
ECR	Elastic Container Registry - Repositorio de imagens Docker na AWS
ECS	Elastic Container Service - Servico de orquestracao de containers Docker
Fargate	Modelo serverless do ECS onde a AWS gerencia a infraestrutura de servidores
IAM	Identity and Access Management - Controle de acesso e permissoes na AWS
Multi-AZ	Implantacao em multiplas zonas de disponibilidade para alta disponibilidade
NAT Gateway	Permite que recursos em subnets privadas acessem a internet
OIDC	OpenID Connect - Protocolo de autenticacao baseado em OAuth 2.0
RDS	Relational Database Service - Banco de dados relacional gerenciado pela AWS
SG	Security Group - Firewall virtual que controla trafego de entrada e saida
SSL/TLS	Protocolos de criptografia para comunicacao segura (HTTPS)
Task Definition	Configuracao que define como um container Docker deve ser executado no ECS
VPC	Virtual Private Cloud - Rede virtual isolada na AWS

Tabela 13 - Glossario de termos tecnicos

Fim do Documento

Bennu Finance - Guia de Instalacao e Configuracao AWS - v1.0 - Fevereiro 2026